

Pocketful.

A BAND payments factory.

Two stages built. Every claimed check passed.

2 / 4

completed stages

182

official checks passed

Small scope. Complete stages.

Two independently buildable services, with the earlier stage preserved.

STAGE 1 / 147 checks passed

Payments and settlements

Payments by handle, money requests, split expenses, and settlement behavior.

STAGE 2 / 35 checks passed

Wallet and authorizations

A browser wallet with available funds, reserved holds, expiry, and captures.

Stages 3 and 4 are incomplete and excluded. This entry documents a supervised practice run, with human assistance disclosed.

A Builder needs someone who can say “check that.”

Generic mandates define the roles. The task carries the product specification.

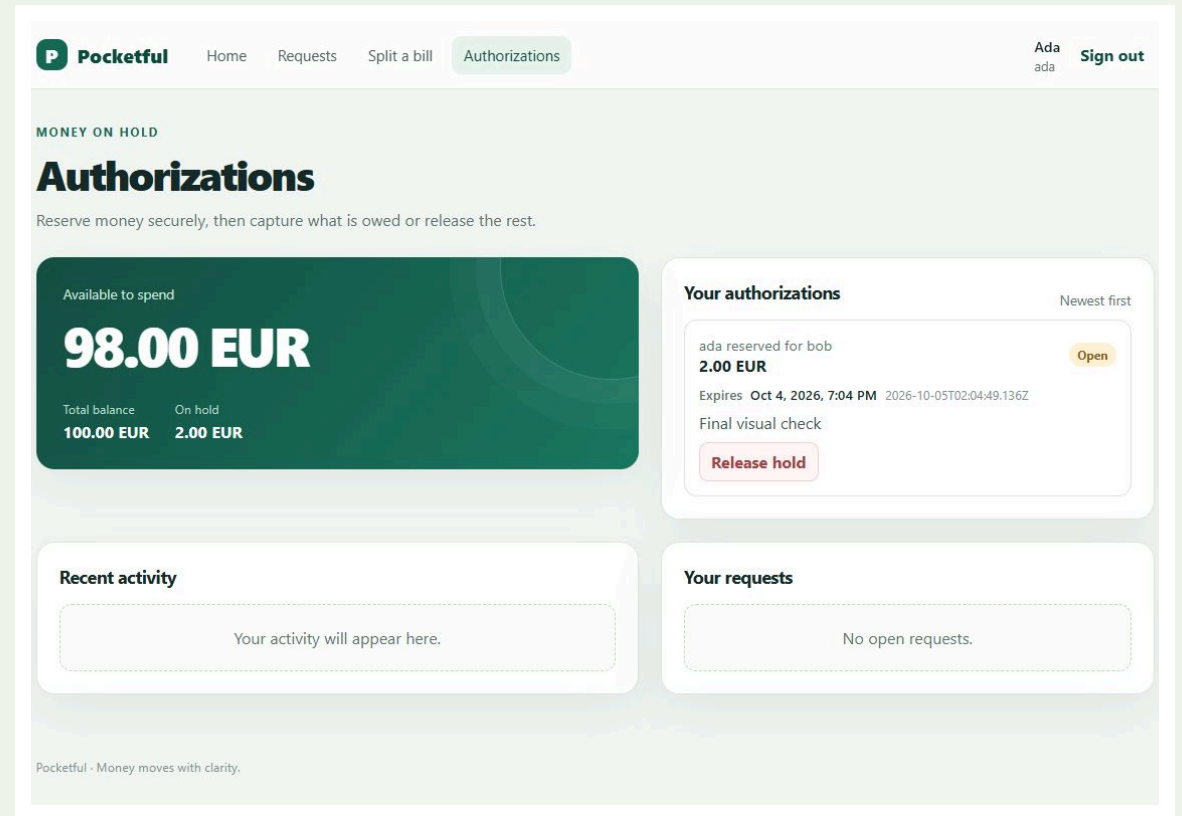
01	02	03	04
Architect	Builder	Verifier	Spec Auditor
Coordinates work and review handoffs.	Owens service implementation.	Checks a fresh clone and probes behavior.	Checks requirements independently.

Money available means money you can actually spend.

Stage 2 makes available funds the primary wallet value.

Total and held balances remain visible.
Open holds reduce availability until capture, release, or expiry.

The app serves its HTML, CSS and JavaScript locally. No external runtime API or asset dependency.



Actual Stage 2 UI / fixture account and balances

182 checks. Zero failures.

147

Stage 1 checks passed

35

Stage 2 checks passed

Fresh clone. Exact committed service revision.

Isolated harness. Both stage suites completed.

Extra probes. Upgrade compatibility, payment retries, and insufficient funds.

Independent approval. Both reviewers approved the final service.

Service revision: 02d0b12a0cb55468540a0a2ff63ee78297a23756

Passing tests didn't finish the UI.

Clipped navigation.

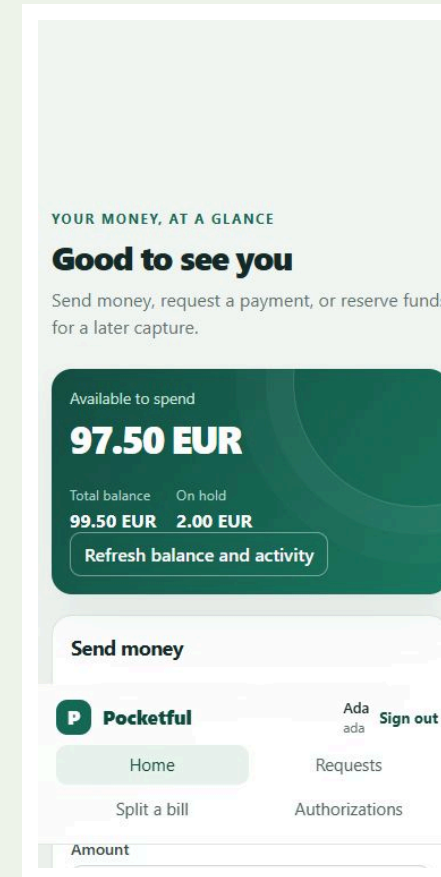
Unclear errors.

Caught. Corrected.

Host desktop and mobile inspection exposed real presentation issues.

Reviewers inspected the corrected screenshots.

A refused payment preserved balances. A successful payment refreshed the receipt. The split screen showed the rounding result.



Actual mobile UI

Follow the evidence.

Build the result

stage-1/ and stage-2/

Dockerfile and RUN.md in each

Earlier stage preserved in Git

Inspect the factory

Generic mandate files

FACTORY.md and design choices

Unchanged full-session room export

Know the limitations

Supervised practice run.

Human execution approvals.

Host debugging and UI checks.

No claim of a fully autonomous judged run. No reliable per-seat cost measurement. Final reviews used existing included Codex and Claude allowances.

We just ran out of time.

We chose to submit the two completed stages with their evidence, rather than imply four-stage completion.

What used the time

Provider and quota troubleshooting.
Execution and review coordination.
Fixing real API and mobile UI findings.
Packaging an authentic room export.

What we protected

Both independently buildable stages.
Fresh-clone checks and exact approvals.
The earlier stage and Git history.
An honest account of human assistance.

This is a completed two-stage practice result. Stages 3 and 4 remain unfinished, and full autonomy is not claimed.

Access and context were part of the engineering problem.

OpenCode cost us time and money

Earlier paid OpenCode experiments consumed money. Go reviewer attempts then hit rolling quota limits. Remaining allowance did not guarantee an available review lane.

Lesson: verify the actual provider route, model, quota window, and spending controls before a run. The final reviews used included Codex and Claude allowances.

Context control was critical

Long sessions and repeated full-spec handoffs made the room harder to follow. Keeping the active revision and outstanding findings clear mattered.

Lesson: use smaller scoped sessions and durable checkpoints. Compact between phases. Preserve the complete specifications required by the handoff rules.

Spending lesson is qualitative: we did not capture reliable per-seat cost measurements. Historical paid experimentation is separate from the no-additional-spending final practice.

A cleaner run, then more stages.

First: make the factory repeatable

Preflight every seat and review lane.
Resolve execution access before dispatch.
Start a fresh autonomous judged run.
Measure usage and record the room from the start.

Then: extend the verified service

Read the complete Stage 3 and 4 specs.
Extend one stage at a time.
Preserve every completed stage.
Require fresh-clone checks and independent approval.

Keep the AGY bridge as an experiment until it proves reliable in a separate preflight. Prepare the public repo, demo URL, and submission assets earlier.

Two stages. 182 passing checks. A record you can inspect.

We built a working payments service and a wallet interface, then kept the review evidence alongside the code.

Exploration: we also prototyped an Antigravity-to-BAND bridge. It is separate from this verified two-stage run and is not claimed as a contributing seat.